

Iron Sentinel:

Incident Response Report

Simulated Multi-Stage Intrusion – IS-2026-001

Logan Dubyak
28 June 2026

1. Executive Summary

On June 20-23, 2026, the Iron Sentinel SOC capstone lab simulated a multi-stage intrusion against a Windows Server 2022 host within an isolated home lab environment. The simulation modeled three distinct attacker techniques: brute-force credential attack, an obfuscated PowerShell execution, and an off-hours account logon. All activity was generated intentionally for testing purposes.

Two of the three techniques were detected and confirmed in real time using custom Splunk detection rules. The third was validated through historical log analysis after a live trigger attempt was disrupted by a VM clock synchronization issue. No data was exfiltrated and no lasting impact occurred, as the exercise was fully contained within the lab.

This report documents the attack timeline, the detection logic that identified each stage, the containment and eradication steps that would apply in a production environment, and the lessons learned from the exercise.

2. Incident Narrative

The simulated attacker first attempted to gain access to the Windows Server 2022 host (WIN-50QKSHQ5M4N) through repeated authentication attempts against a non-existent local account. Six rapid login attempts were made in succession, each failing with an “unknown user name or bad password” response. This pattern repeated multiple times across the test window, generating 84 total failed logon events.

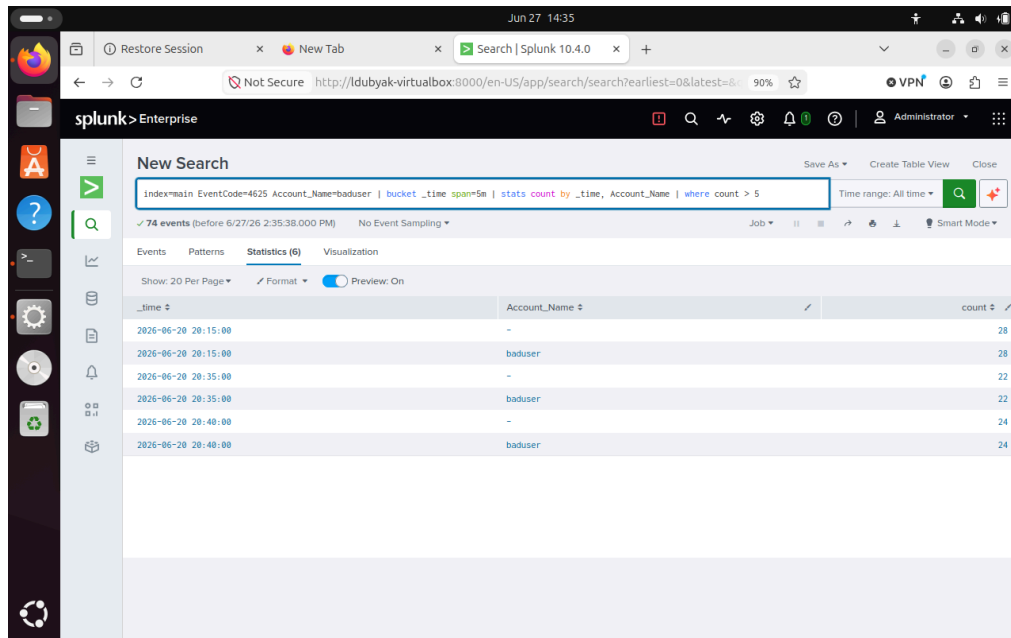


Figure 1. Splunk detection rule confirming the brute force pattern — three 5-minute windows (28, 22, and 24 failed attempts) exceeding the 5-attempt threshold for account "baduser."

Following the brute-force activity, an obfuscated PowerShell command was executed on the same host. The command used a hidden window, a no-profile flag, and a Base64-encoded payload. A combination used by attackers to avoid detection and hide their intentions from anyone monitoring the process list or command history.

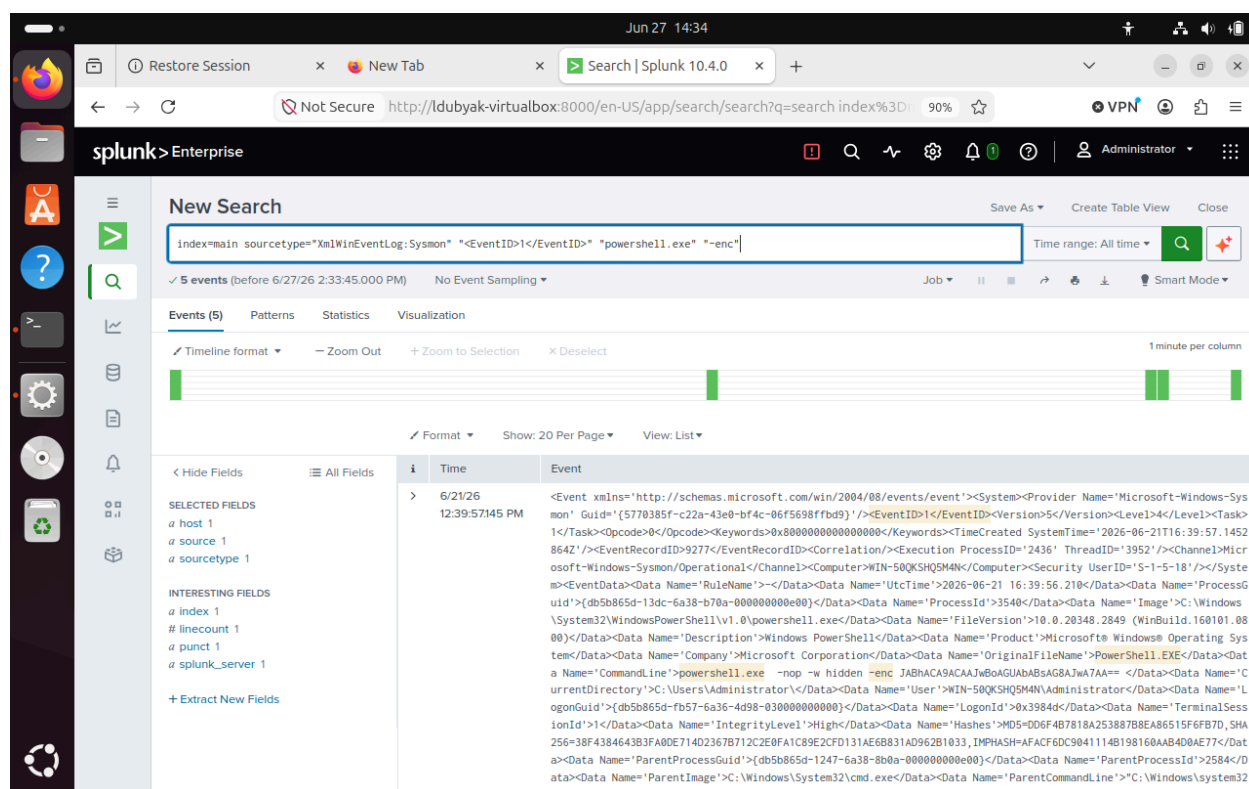


Figure 2. Sysmon Event ID 1 capturing the obfuscated PowerShell execution, with the full command line showing the -nop -w hidden -enc flag combination.

Finally, the simulation modeled an off-hours account logon, representing a scenario where a valid account is used outside of normal working hours. This is a common indicator for a compromised account, credentials, or insider misuse. While a live off-hours login could not be cleanly captured due to the lab environment's conflict with the clock. The underlying detection logic was verified against the full set of legitimate logons recorded during the exercise, confirming it would correctly flag this behavior in a live environment.

All three techniques were captured by Sysmon and Windows Security auditing, forwarded to Splunk via the Universal Forwarder, and indexed for analysis. Full technical details including commands, timestamps, and raw log evidence is documented in the accompanying Attack Timeline document.

3. Techniques and Detection Summary

Technique	MITRE ID	Detection Rule	Result
Brute Force (NTLM)	T1110	Failed login Brute Force Detection	Confirmed- 3 Windows exceeded threshold
PowerShell Obfuscation	T1059.001	PowerShell Obfuscation Detection	Confirmed- command line captured
Off-Hour Login	T1078	Off-hour Login Detection	Logic Verified against 720 events

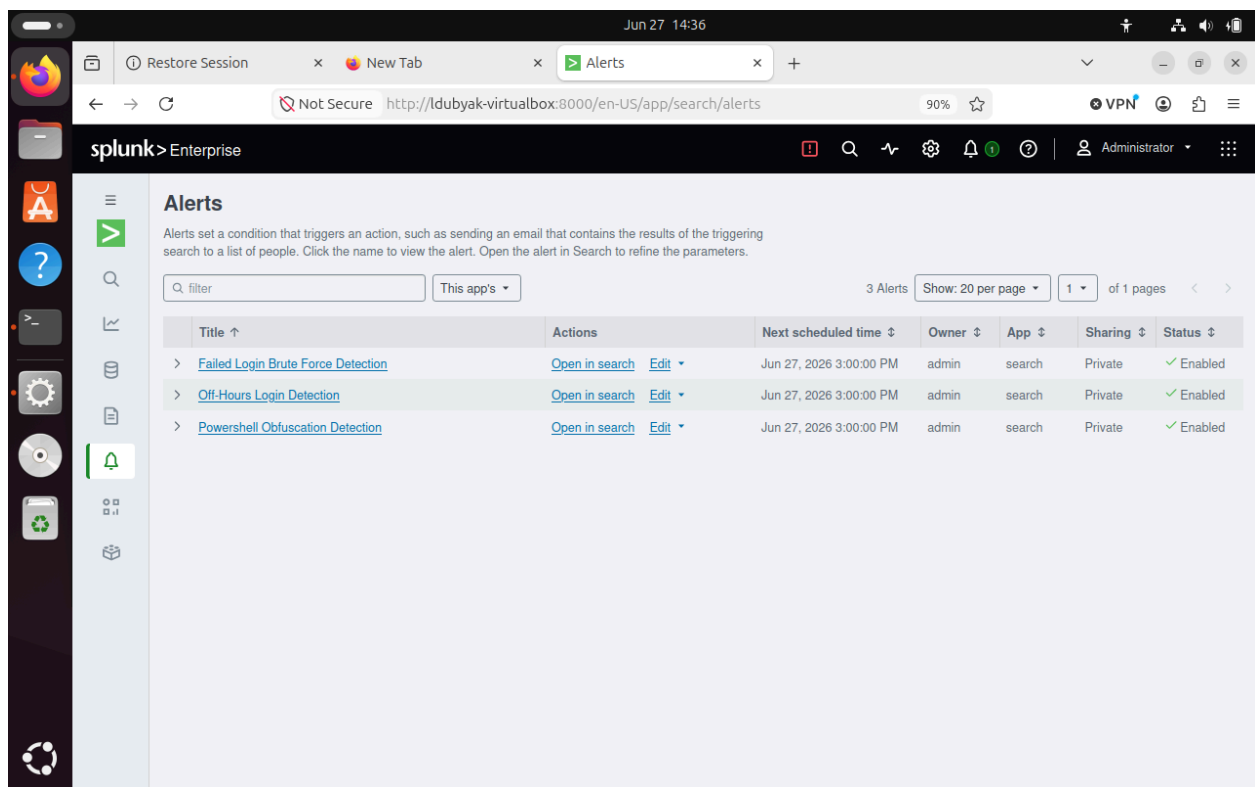


Figure 3. All three Splunk detection rules are confirmed active and enabled in the Iron Sentinel SIEM environment.

3. Terminate any suspicious PowerShell processes still running, and capture the full process tree and command line for analysis before termination.
4. Review authentication logs for the same source IP or workstation across all other hosts to confirm whether the activity is isolated or part of a broader campaign.

6. Eradication and Recovery

5. Reset credentials for any account involved in the brute force or off hours logon activity.
6. Review and tighten PowerShell execution policy and enable Constrained Language Mode to reduce the impact of future obfuscated script execution.
7. Confirm no persistence mechanisms (scheduled tasks, registry run keys, new services) were created during the incident window.
8. Restore the host from a known-good snapshot if any unauthorized changes are identified; in this lab, the pre-test VM snapshot served this purpose.
9. Re-enable network access only after confirming the host is clean and all detection rules are active and current.

7. Recommendations

- Lower the brute-force detection threshold from 5 to a smaller number, such as 3, within the same 5-minute window to reduce attacker dwell time before an alert is made.
- Extend the PowerShell detection rule to also flag Invoke-Expression, IEX, and DownloadString patterns, which

commonly are used with obfuscated commands in real-world attacks.

- Schedule a recurring health check on the Splunk Universal Forwarder connection, since a multi-day forwarding gap went unnoticed during this exercise and could mask a real incident in production.
- Document and test off hours and other time based detection rules against historical data on a regular time frame if live-fired trigger testing remains difficult to safely or reliably perform.

8. Lessons Learned

- A multi-day gap in forwarding went unnoticed until manually checked. Forwarder health should be verified at the start of every monitoring session and not assumed.
- Raw XML Sysmon data behaves differently in Splunk than the extracted fields. Searches require raw test matching rather than standard field syntax unless field extraction is explicitly configured. This is a meaningful operational detail for any SOC onboarding a new Sysmon data source.
- Not every detection rule requires a live-fired trigger to be considered validated. Verifying rule logic against historical data is a legitimate and often necessary method, particularly for time-boundary or low-frequency conditions.
- Third-party offensive security tooling (Atomic Red Team) can be flagged by endpoint antivirus as malicious by design, since its test payloads mimic real attacker behavior. Manually scripting equivalent test commands avoided the issue and still allowed for testing the detection rule logic.

